Hitec University Taxila, Department of Computer Engineering

Lab Manual for Mobile Application Development

Lab-6 List View and BaseAdapter

Contents

1 Introduction

Android ListView is a view which groups several items and display them in vertical scrollable list**.** The list items are automatically inserted to the list using an Adapter that pulls content from a source such as an array or database**.** Android provides several subclasses of Adapter that are useful for retrieving different kinds of data and building views for an AdapterView (i.e. ListView or GridView) The common adapters are

- ArrayAdapter
- BaseAdapter

BaseAdapter is a common base class of a general implementation of an Adapter that can be used in ListView, GridView, Spinner etc. Whenever you need a customized list in a ListView or customized grids in a GridView you create your own adapter and extend base adapter in that. Base Adapter can be extended to create a custom Adapter for displaying a custom list item.

2 Activity Time boxing

Table 1: Activity Time Boxing

Task#	Activity Name	Activity time	Total Time
1	Introduction	60mins	60min
2	Walk Through Task	60mins	60min
3	Evaluation Task	60min	60min

3 Objectives of the experiment

- To get basic understanding List View and Base Adapter
- To be able to use List View and Base Adapter in android applications.
- To get to know about the benefits related to List View.

4 Walk through Task

Creating New Activity

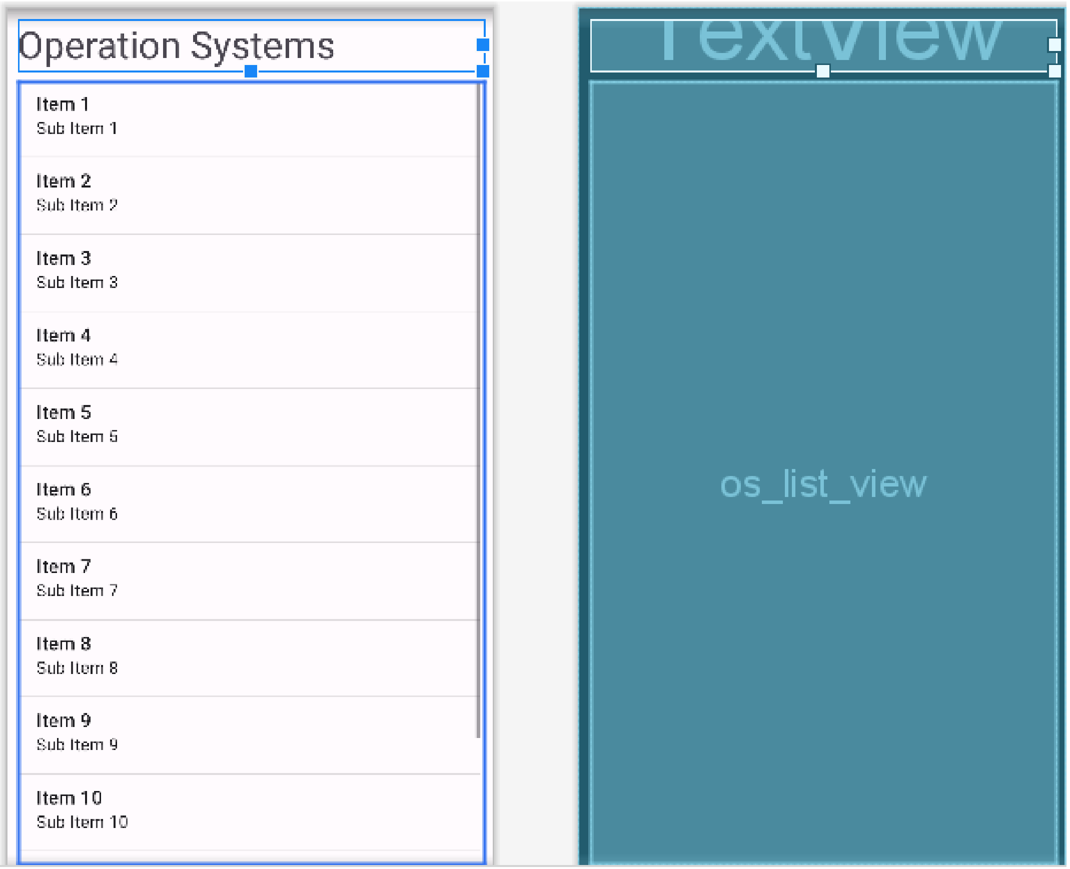
Step 1: Create Activities:

Create a new project just like you have created in your previous labs and create one more empty activity.

Step 2: Design XMLs of Activity:

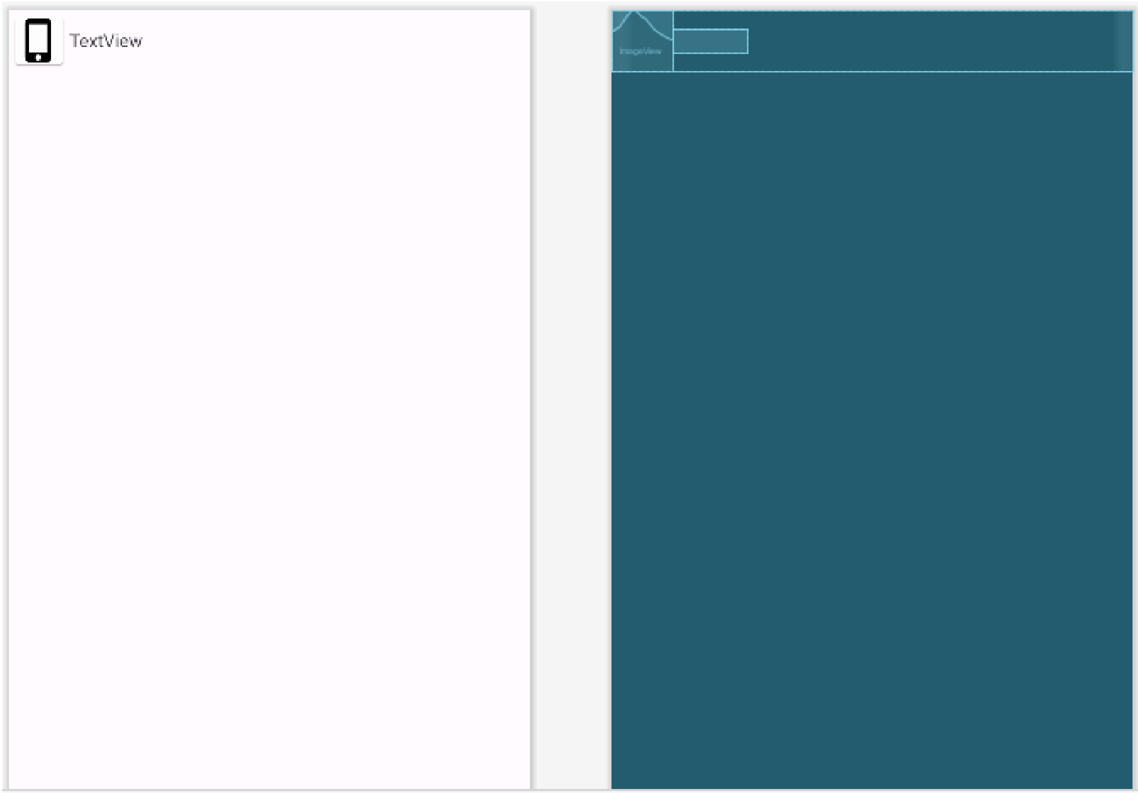
Designs of activitiy are given below, just use the given xml code which is given in this step.

activity_operating_system.xml




```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    android:orientation="vertical"
    android:padding="@dimen/view_margin"
    tools:context="OperatingSystemCustomAdapter.OperatingSystemActivity">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="32sp"
        android:layout_marginBottom="@dimen/view_margin"
        android:text="Operation Systems">

    </TextView>

    <ListView
        android:id="@+id/os_list_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

</LinearLayout>
```

Step 3: Design os_item_view.xml:




```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <ImageView
        android:id="@+id/os_image_view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@mipmap/os_ios" />

    <TextView
        android:id="@+id/os_name_text_view"
        android:layout_gravity="center"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="TextView" />

</LinearLayout>
```

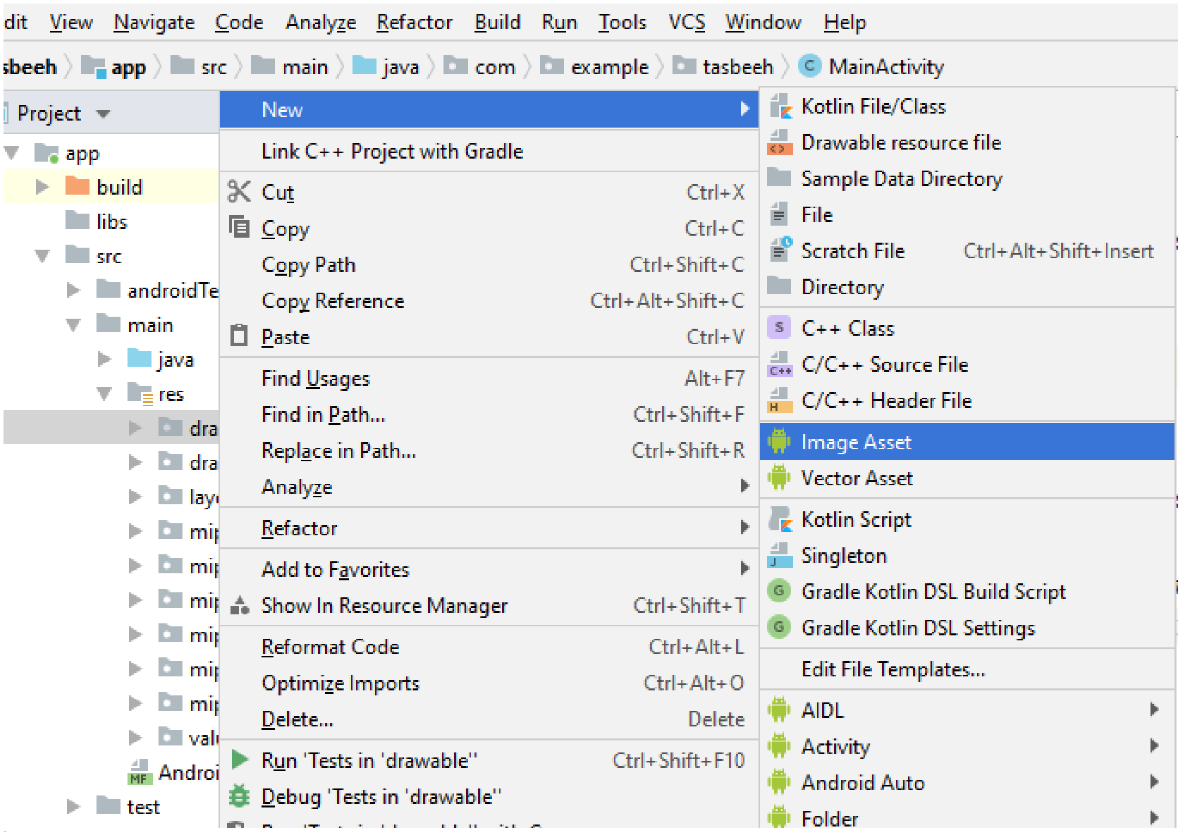
Step 4: Write Custom Adapter Class

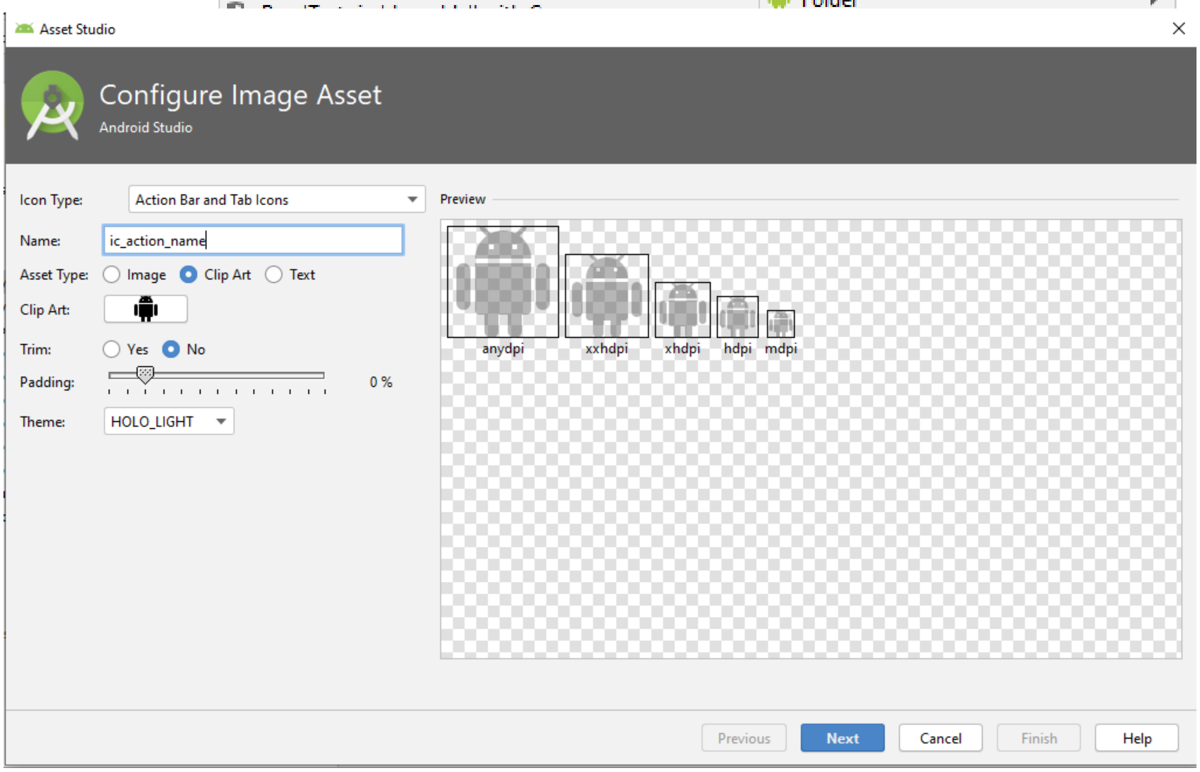
```
public class OperatingSystemCustomAdapter extends BaseAdapter {
    ArrayList<String> osAL;

    ArrayList<Integer> osIocnAL;

    LayoutInflater inflater;
    public OperatingSystemCustomAdapter(Context context, ArrayList<String> osAL,
```

```
                                        ArrayList<Integer> osIocnAL) {
        this.osAL = osAL;
        this.osIocnAL = osIocnAL;
        inflater = LayoutInflater.from(context);
    }


    @Override
    public int getCount() {
        return osAL.size();
    }

    @Override
    public Object getItem(int position) {
        return null;
    }

    @Override
    public long getItemId(int position) {
        return 0;
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        convertView = inflater.inflate(R.layout.os_item_layout, null);

        ImageView countryFlagImageView =
convertView.findViewById(R.id.os_image_view);
        TextView countryTV = convertView.findViewById(R.id.os_name_text_view);

        countryFlagImageView.setImageResource(osIocnAL.get(position));
        countryTV.setText(osAL.get(position));

        return convertView;
    }
}
```

Step 5: Add images in Android Studio Project

To add new images in drawable folder, use image assets to add new image from repository or from local storage





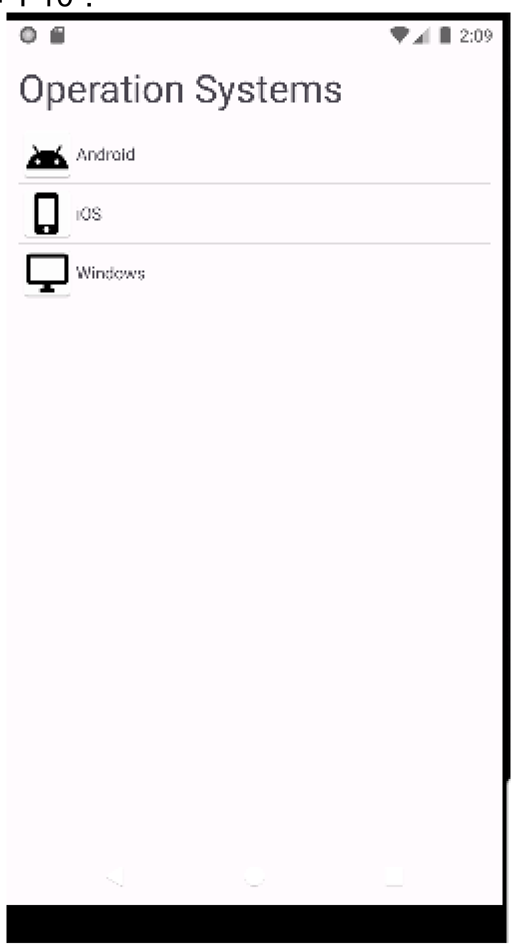
Step 6: Write adapter in Main Activity

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_operating_system);

        ArrayList<String> osAL = new ArrayList<String>();
        osAL.add("Android");
        osAL.add("iOS");
        osAL.add("Windows");

        ArrayList<Integer> osIconAL = new ArrayList<Integer>();
        osIconAL.add(R.mipmap.os_android);
        osIconAL.add(R.mipmap.os_ios);
        osIconAL.add(R.mipmap.os_windows);

        OperatingSystemCustomAdapter adapter = new
OperatingSystemCustomAdapter(this, osAL,
                osIconAL);

        ListView listView = findViewById(R.id.os_list_view);
        listView.setAdapter(adapter);
    }
}
```

Step 7: Now Run Project

Click “Run” or press “Shift + F10”.



5 Home Task

Design a list view to show country flag, name, and their capital city as shown in the figure below. Add at least three countries into the list.